# Grammar-Grounded Form Generation with QLoRA and Compiler Feedback

Final-Year Project Report

Rishi Raval
Roll No.: 1AUA23BCS155
Adani University
Faculty: Krupali Donda

October 30, 2026

**Abstract**

This project studies a constrained generative-AI task: translating natural-language form requirements into Forml, a domain-specific language (DSL) used by Formix. A Qwen2.5-Coder-1.5B-Instruct causal language model is adapted with parameter-efficient QLoRA fine-tuning on compiler-checked synthetic examples. A hand-written Forml compiler acts as the validity oracle during dataset construction, evaluation, and an optional generate–compile–repair loop. On a held-out, scenario-based set of 420 examples, the base model compiled 1.19% of first outputs and 3.81% after two repair turns; the adapted model compiled 73.33% on its first output and 73.33% after repair. The study also reports expected-field coverage, training configuration, compiler role, limitations, and reproducibility details. The benchmark is grammar-grounded and synthetic, so these results measure performance on this project benchmark rather than broad real-world form-generation ability.

## 1 Problem and project goal

Formix lets users describe forms as code. Forml is its small, structured DSL for fields, validation, layout, conditional behavior, and form actions. The project goal is to adapt a code-capable language model to translate a human request into complete Forml source, then rely on the actual Formix compiler to decide whether the output is valid. This separates two roles: the neural model proposes code; deterministic compiler software checks it.

The work is a focused generative-AI application rather than training a foundation model from scratch. It includes dataset generation, parameter-efficient fine-tuning, a compiler-based evaluation, and an interactive local/Kaggle demonstration. The model is not deployed as a live Formix website service in this project scope.

## 2 Forml and its EBNF grammar

Forml is a domain-specific language (DSL): a compact language designed for a specific job, here describing forms. Its EBNF specification describes which token sequences are legal. In EBNF, a production defines a rule, alternatives are separated by |, square brackets mean optional, and braces mean repetition (zero or more times).

For example, the core shape is:

```
form = "form" STRING "{" { statement } [ action_block ] "}" ;
statement = field | section | repeat_group | conditional
          | group_use | layout_block ;
field = "field" IDENTIFIER ":" type [ ui_block ]
        [ validation_block ] [ compute_block ] [ trigger_block ] ;
```

This says that a form starts with the keyword form, has a quoted name, and contains zero or more statements, optionally followed by an action block. A field has a name and type, with optional user-interface, validation, computation, or trigger details. Other grammar rules describe pages, reusable groups, upload/select fields, expressions, conditions, and actions. The full grammar is maintained in formlcompiler/EBNF_grammar.md (version 1.1); this report shows only the idea, not every production.

Listing 1: Illustrative Forml output from the local demonstration.

```
form "Event Signup Form" {
  field name: text
    ui { label: "Name" }
    validate { required }
  field email: email
    ui { label: "Email address" }
    validate { required }
}
```

## 3 System design

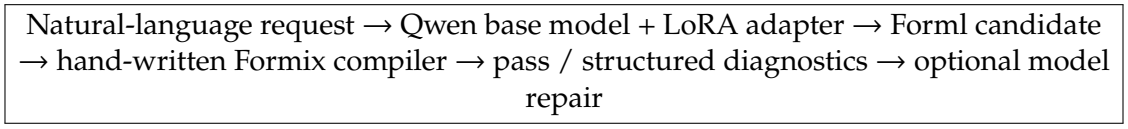



Figure 1: Generation and compiler-feedback validation flow. The compiler, not a hosted language model, determines validity.

The compiler is a hand-written C++ front end with a lexer, recursive-descent parser, semantic analysis, and JSON serialization. It reports syntax and semantic diagnostics. During generation, the model may receive those diagnostics and the current source for up to two repair turns. Evaluation separately records first-pass validity and validity after repair. A compiler pass establishes acceptance under the implemented language rules; it does not establish that the form is attractive, complete for every unstated requirement, or safe for production data handling.

## 4 Base model and architecture

The base model is Qwen/Qwen2.5-Coder-1.5B-Instruct, an instruction-tuned causal language model in the Qwen2 family. Its published model configuration identifies the architecture as Qwen2ForCausalLM, with 28 transformer decoder layers, hidden size 1,536, 12 attention heads, 2 key/value heads (grouped-query attention), intermediate size 8,960, a 151,936-token vocabulary, and a configured context length of 32,768 tokens [1]. The implementation uses the model's chat template to format system and user instructions.

The model has two components in this experiment: the frozen pretrained base weights and a comparatively small learned LoRA adapter. The adapter is not a standalone language model. In training, the base weights are loaded in 4-bit NF4 form with double quantization; the training

computation uses FP16. In the Kaggle evaluation, base-model weights are loaded in FP16 for batched inference. The one-prompt local demo loads the base in 4-bit to reduce memory needs.

## 5 Dataset construction

The dataset generator creates paired natural-language requests and Forml completions, plus repair examples. It produces 1,200 generation examples and attempts to create 1,200 related repair examples. Before inclusion, a target completion must pass the native Formix compiler. A repair input is deliberately corrupted and must fail compilation; its target is the corresponding valid source, and the prompt includes real compiler diagnostics.

The generated records include task type, scenario family, expected field names, and grammar feature labels. The split used in this run contains 1,782 training examples, 198 validation examples, and 420 test examples (2,400 total). Scenario families are held out for testing to reduce direct leakage of recipe templates across splits. The generator uses seed 42. The exact grammar file is hashed into the generated dataset summary when the dataset is created.

**Dataset limitation.** These are synthetic, grammar-grounded examples generated from templates and compositional field recipes. A scenario split reduces near-duplicate leakage but does not make the test requests human-authored or independent of the generator's design. This should be stated when interpreting the benchmark. A stronger next experiment would add a manually written prompt set and have reviewers score requirement satisfaction.

## 6 Fine-tuning method

We use supervised fine-tuning with Low-Rank Adaptation (LoRA), quantized using QLoRA. The base model is frozen; trainable low-rank matrices are inserted into selected attention and feed-forward projections. This reduces trainable parameter count and GPU memory use while teaching the model the Forml output convention and compiler-oriented repair task. The adapter targets $\mathrm{q_proj}$, $\mathrm{k_proj}$, $\mathrm{v_proj}$, $\mathrm{o_proj}$, $\mathrm{gate_proj}$, $\mathrm{up_proj}$, and $\mathrm{down_proj}$.

Table 1: Training configuration recorded by the project scripts and adapter.

| Item | Value |
|---|---|
| Base model | $\mathrm{Qwen/Qwen2.5\text{-}Coder\text{-}1.5B\text{-}Instruct}$ |
| Method | Supervised fine-tuning with LoRA / 4-bit QLoRA (NF4, double quantization) |
| LoRA rank $r$ / alpha | 16 / 32 |
| LoRA dropout | 0.05 |
| Trainable target modules | q/k/v/o and gate/up/down projections |
| Training precision | FP16 compute; 4-bit quantized frozen base |
| Sequence length | 2,048 tokens |
| Epochs | 3 |
| Learning rate | $2 \times 10^{-4}$ |
| Per-device batch / gradient accumulation | 1 / 8 |
| Optimizer / schedule | paged AdamW 8-bit / cosine, 3% warmup |
| Gradient checkpointing | Enabled |
| Random seed | 42 |
| Runtime / device reported | 3,064.21 seconds (about 51.1 minutes) / Tesla T4 |
| Final training loss | 0.01193 |

The aggregate training loss is not a held-out quality score. Per-step loss history and final validation loss were not retained in the run metadata, so no learning curve or validation metric is reported.

## 7 Evaluation protocol and metrics

The evaluation compares the base model and the adapted model on the same 420 held-out examples. Both use FP16, batch size 8, and up to 350 generated tokens per pass. For each example, generated source is sent to the native Formix compiler. Invalid outputs are offered up to two compiler-feedback repair turns. The evaluator records:

- **First-pass compile rate**: fraction of initial generations accepted by the compiler.
- **Compile rate after repairs**: fraction accepted after zero to two repair turns.
- **Expected-field coverage**: overlap between expected field identifiers and identifiers extracted from generated text, averaged over examples. This is a lightweight lexical metric, not semantic equivalence.
- **Repair success on initially invalid examples**: initially invalid outputs that become valid after repair, divided by all initially invalid outputs.

Table 2: Held-out benchmark results, 420 examples per model. Percentages are derived from the saved run summary.

| Model | First pass | After repair | Field coverage | Repair rescue |
|---|---|---|---|---|
| Base model | 1.19% | 3.81% | 29.61% | 2.65% |
| Fine-tuned | 73.33% | 73.33% | 93.10% | 0.00% |

The adapted model's first-pass compile rate is 72.14 percentage points higher on this benchmark. The fact that its repair stage does not improve aggregate validity is a real finding to discuss: the current repair prompts/training examples did not rescue its remaining invalid outputs in this evaluation. The base model's absolute repair rescue is 2.65% of its initially invalid examples (11 of 415); the adapted model rescued 0 of 112.

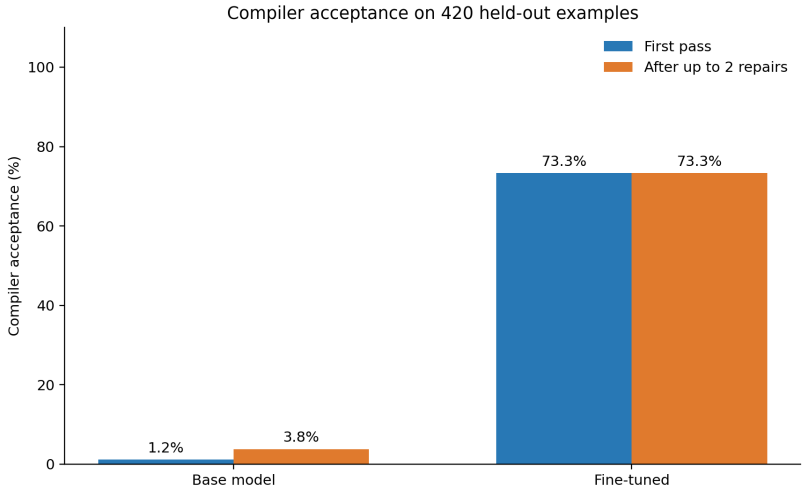



Figure 2: Compiler acceptance before and after up to two repair turns.

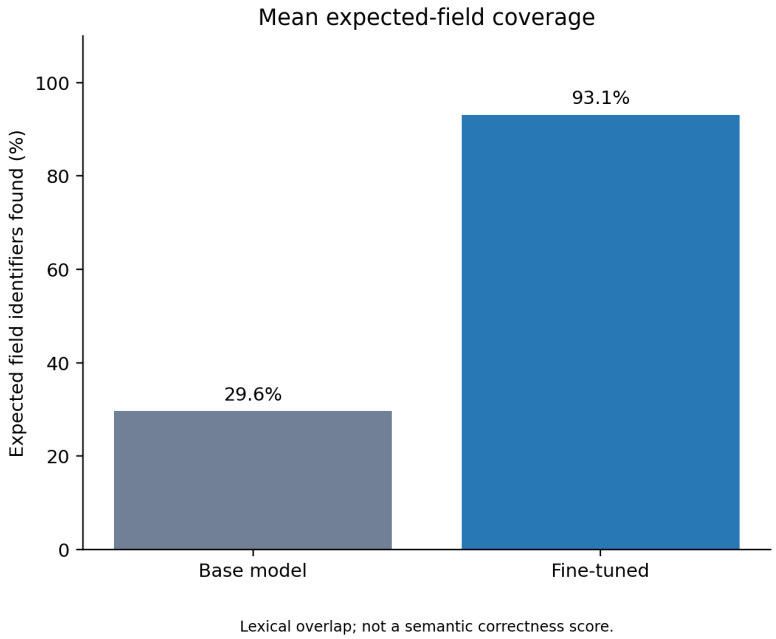



Figure 3: Mean expected-field coverage. This is lexical identifier overlap, not a semantic correctness measure.

## 8 Demonstration

The project can be demonstrated in a Kaggle notebook or a CUDA-enabled local environment. A user enters a natural-language request; the adapter-backed model returns Forml; the native compiler prints PASS or diagnostics. If the first result fails, the demo can feed diagnostics back for up to two repair attempts. The demonstrated example, "Create an event signup form with name, email, and dietary preferences," produced a form with name, email, and dietary preference fields, and the compiler returned PASS.

This is evidence that the end-to-end inference and compiler integration work for that example. It is not evidence that every generated form is correct; the held-out benchmark is the quantitative evaluation.

## 9 Reproducibility and artifacts

The repository contains the dataset generator, training script, evaluator, local demo, compiler, adapter configuration, and training metadata. Reproduction requires the named base model, learned adapter, evaluation JSON, and dataset summary with the grammar hash, seed, and held-out scenarios. The script docs/genai-report/make_figures.py generates PNG charts and CSV summaries from the evaluation JSON. These artifacts and compiler build instructions should accompany the submission.

## 10 Limitations and next steps

The benchmark is synthetic and limited to one DSL and one generated test split. Field coverage is based on a regular expression and does not verify field type, labels, validation rules, conditional behavior, or whether the form fully meets a human request. Compiler acceptance verifies implementation-defined syntax and semantics, not user intent. Only one fine-tuning configuration and one base model were evaluated. The aggregate training loss is unusually low relative

to this small synthetic corpus and may reflect the templated nature of the task; it must not be presented as evidence of generalization by itself. Repair success of zero for the adapted model indicates a weakness worth investigating, rather than hiding.

Recommended next steps are to collect a small human-authored prompt set, manually rate requirement satisfaction, report task/scenario-stratified metrics, compare multiple random seeds or a no-repair ablation, and improve repair-specific evaluation. Those experiments should be clearly marked as future work unless completed.

## 11 Conclusion

This project demonstrates a complete constrained-generation workflow: a pretrained code model is adapted with QLoRA on compiler-checked DSL data, and a hand-written compiler provides deterministic validation and feedback. On the current held-out synthetic benchmark, fine-tuning greatly improves compile acceptance and expected-field coverage over the base model. The results are promising for the Forml task, but their scope is bounded by synthetic data and by the metrics' limited measures of user-intent satisfaction.

## References


[1] Qwen Team, *Qwen2.5-Coder-1.5B-Instruct model configuration*, Hugging Face model repository. Available at https://huggingface.co/Qwen/Qwen2.5-Coder-1.5B-Instruct/blob/main/config.json (accessed 24 October 2026).

[2] E. J. Hu et al., "LoRA: Low-Rank Adaptation of Large Language Models," arXiv:2106.09685, 2021. https://arxiv.org/abs/2106.09685.

[3] T. Dettmers et al., "QLoRA: Efficient Finetuning of Quantized LLMs," arXiv:2305.14314, 2023. https://arxiv.org/abs/2305.14314.